

IDONTKNOWITSNAME

Anonymous Author(s)

Abstract

LLM agent frameworks increasingly expose model invocations through structured programming abstractions, with framework runtimes lowering program-level constructs into concrete prompts. We observe that these programmatically generated prompts are inherently heterogeneous: their components exhibit different sharing scopes, lifetimes, and evolution patterns across invocations. However, existing serving systems treat prompts as opaque text, losing this structure and missing opportunities for more effective KV-cache reuse and management.

We present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME’s compiler pass performs static analysis and compiler-level optimizations to expose binding lifetimes, expose application-level structure and proactively re-layout prompts to improve cache reuse. The serving runtime exploits the heterogeneity information to optimize KV-cache behavior across requests by prefetching, protection and eviction, according to the predicted future utility of cache entries. We implemented IDONTKNOWITSNAME on SGLang and evaluated it against multiple established serving systems. Our results shows that IDONTKNOWITSNAME.....

1 Introduction

LLM agents rarely relies on a single model requests. Instead, they typically issue a series of dependent LLM calls to accomplish a task. Intermediate results evolve and stream across these calls as the agent executes, resulting in heavy KV-cache pressure. Agents’ task performance is often bottlenecked by the efficiency of KV-cache management. Repeatedly processing and retaining these overlapping contexts can impose substantial computation and memory overhead on the serving system. Efficient KV-cache management is therefore critical to improve the performance of serving LLM agents. Existing agent serving systems, such as Pythia [15], CacheScout [16], PBKV [17], and KVFlow [12], optimize KV-cache management by exploiting future invocations and workflow information. They exploit these knowledge to guide KV-cache retention, eviction and prefetching.

Meanwhile, agent frameworks increasingly expose LLM calls as programming abstractions rather than manual prompt construction. ByLLM [2] introduces a language-level abstraction that delegates function execution to an LLM, with the runtime using semantic information from the code like function signatures and parameter names to construct model requests. NVIDIA Object-Oriented Agents (NOOA) [5] similarly embeds LLM-driven behavior into native Python objects and translates program-level declarations into prompts

and model interactions. DSPy [9] represents LM invocations through declarative signatures and modules, while the framework automatically generates a concrete prompt. This trend even extends beyond hand-written agent code. SIGIL [3] compiles natural-language skill documents into programs with explicit control flow and typed LLM calls, which the runtime lowers into prompts. Across these systems, LLM behaviors are specified using structured programming constructs, while framework runtimes translate those constructs into concrete prompts and model requests.

These two trends expose an substantial granularity mismatch. Agent-serving systems optimize KV-cache management primarily at the granularity of requests and call sequences, whereas such LLM-as-programming-abstractions frameworks assemble each request from multiple program components. Therefore, treating such programmatically generated prompts as monolithic requests may lose valuable information for efficient LLM serving.

We observed that these programmatically generated LLM requests are inherently heterogeneous. Components within a generated prompt can have different sharing scopes and lifetimes. For example, some components may remain fixed across invocations, some are shared across call sites, and some can grow or be replaced as execution progresses. Figure 1 shows an example, the four bindings of `assess_evidence` have distinct lifetimes and evolution patterns. For example, evidences are session-scoped and accumulate over time, while `search_results` is just a single-shot result. In this case, placing evidences after `search_research` blocks KV-cache reuse of evidences across requests. Then model engine needs to recompute for these tokens, wasting scarce computational resource. This kind of differences in sharing scope and lifetime requires a more fine-grained management of KV-cache entries.

However, existing agent serving system are blind to this heterogeneity as they overlook the granularity inside the prompt. Mitigating this gap introduces two challenges. First, heterogeneity is not always explicit in the prompt, their sharing scope and lifetime are closely bonded with the program-level structure. Second, context scale and workflow evolution are often non-deterministic due to the inherent uncertainty of LLMs. Proactively management of KV-cache requires balance the trade-off between the cost of memory usage and the benefit of cache reuse.

To close this gap, we propose IDONTKNOWITSNAME, a compiler-server co-design that exploits prompt heterogeneity to achieve more efficient KV-cache management. IDONTKNOWITSNAME’s compiler pass performs static analysis to identify heterogeneous binding evolution patterns and application-level structure, and proactively re-layouts prompts

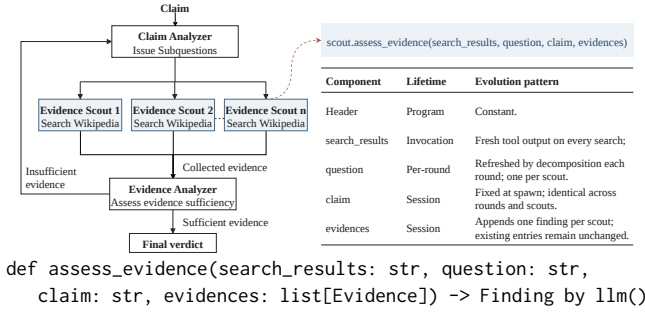


Figure 1. Heterogeneity within a HoVer-style [8] Fact Check Agent’s generated prompt for the summary-making function. `assess_evidence(search_results, question, claim, evidences)` evaluates the sub-question against the accumulated evidence and search results, then answers it.

to improve cache reuse. The serving runtime exploits the application-level structure and binding evolution patterns to optimize KV-cache behavior across requests by prefetching, protection, and eviction, according to the predicted future utility of cache entries. We implemented `IDONTKNOWITSNAME` on SGLang [18]. Our evaluation shows that `IDONTKNOWITSNAME`...

In summary, we make the following contributions.

- We identify the inherent heterogeneity in programmatically generated prompts and its implications for serving.
- We design and build `IDONTKNOWITSNAME`, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management.
- We conduct comprehensive evaluations of `IDONTKNOWITSNAME` against multiple established serving systems, demonstrating that `IDONTKNOWITSNAME` achieves up to...

2 Background

2.1 From program constructs to prompts

Agent frameworks separate the declaration of an LLM operation from the construction of its input. In ByLLM, a developer supplies a function signature, types, and semantic descriptions, and the runtime constructs the prompt and parses the response [2]. DSPy expresses operations through signatures with named input and output fields, which adapters translate into model messages [9]. We use *call site* to denote an identifiable LLM operation in an application, and *binding* to denote a named input together with its value in a particular invocation. A *session* is one execution of a user-level task, potentially involving repeated calls to several sites and intervening tool executions.

ByLLM-style binding layout (abridged)

```
Coder.implement(spec, plan, advice, attempts)
spec = <function specification>
plan = <implementation plan>
advice = <feedback for this attempt>
attempts = <accumulated attempt records>
```

DSPy ChatAdapter field layout (abridged)

```
[[ ## spec ## ]] <function specification>
[[ ## plan ## ]] <implementation plan>
[[ ## advice ## ]] <feedback for this attempt>
[[ ## attempts ## ]] <accumulated records>
```

Figure 2. Schematic rendering of the same coding inputs using named bindings and delimited fields. System instructions, schemas, and output instructions are omitted. The field values have different lifetimes despite occupying the same message.

For the template-based requests considered here, lowering combines framework instructions, call-site metadata, serialized bindings, and output-format instructions. The metadata can include a qualified function name, parameter types, and field descriptions. ByLLM-style rendering labels argument values by name; DSPy’s ChatAdapter uses explicit field delimiters [4]. Figure 2 illustrates these two forms. Their concrete message layouts differ, but both retain boundaries between named values and surrounding instructions. With the declaration, adapter configuration, and rendering rules fixed, repeated invocations can retain a common skeleton while their binding values change.

2.2 Prefix reuse and cache placement

An autoregressive transformer processes input tokens during *prefill*, producing per-layer key and value tensors, and then generates output tokens during *decode*. Keeping these tensors avoids recomputing earlier tokens during generation. PagedAttention, introduced in vLLM, manages KV memory in blocks to reduce fragmentation and support sharing [10]. SGLang’s RadixAttention indexes cached token sequences in a radix tree to reuse common prefixes across requests [18].

Under ordinary causal attention, a token’s cached state depends on its preceding context. Consequently, exact prefix caching requires matching tokens and compatible model and positional settings: identical text after a changed prefix cannot generally reuse its old KV state. A cache lookup can reuse the resident portion of the longest common token prefix; the remaining input requires prefill. This is a constraint on the serialized token sequence, independent of whether two later fields carry the same application-level value.

Cache availability imposes a second constraint. Hierarchical caching retains KV outside GPU memory and transfers it back when needed. CachedAttention combines hierarchical storage with overlapped loading and saving for multi-turn conversations [6]; Mooncake organizes distributed KV storage around disaggregated prefill and decode resources [13].

In the device–host setting used by IDONTKNOWITSNAME, a device hit avoids both transfer and recomputation, a host hit requires promotion, and a miss in both tiers requires re-computing the missing prefix. Chunked prefill can interleave this computation with decoding, as in Sarathi-Serve [1], but does not eliminate the work.

These costs affect both per-call time to first token (TTFT) and session job completion time (JCT). A session can revisit a call site after search, testing, or another LLM operation, during which competing requests may displace its cache. Efficient reuse therefore depends on both the prefix exposed by prompt construction and the availability of that prefix when execution returns.

3 Motivation

3.1 Lifetime heterogeneity within one prompt

The coding example in Figure 2 illustrates why treating an entire prompt as one reuse unit loses useful information. Consider an iterative implementation task in which spec and plan remain fixed, advice is replaced after a failed test, and attempts accumulates earlier implementations and test results. These fields belong to the same invocation, yet their next values have different relationships to the current ones. The specification and plan can be copied, part of the attempt history remains known, and the next feedback is unavailable until testing completes. Framework instructions may persist even across sessions, whereas the specification is private to one task.

The fact-checking example in Figure 1 has the same structure: the claim persists across a verification session, evidence accumulates, and a new sub-question or search result can replace the corresponding input. Moreover, the same claim or evidence may be passed between call sites. This creates two distinct dimensions of heterogeneity: how a binding evolves over time, and where its value is shared. A field can be stable within one session without being a program-wide constant, and it can be shared across sites without remaining unchanged forever. These properties must be inferred from values and execution history; a parameter name or type alone does not establish them.

3.2 Declaration order hides reusable content

Frameworks organize fields to express an operation’s interface. That order need not expose the longest reusable token prefix. Suppose the coding request is rendered as

$$P_t = H\|S\|L\|A_t\|E_t\|R,$$

where H contains fixed leading instructions and metadata, S is the specification, L is the plan, A_t is current advice, E_t is the attempt history, and R contains trailing instructions. Each field includes its label and delimiters. When advice changes, prefix matching stops within A_t even if most of E_t remains identical. Keeping the entire previous request in

GPU memory cannot recover this lost reuse: the old history tensors were computed under the old advice.

Placing the attempt history before advice instead gives

$$P'_t = H\|S\|L\|E_t\|A_t\|R.$$

If consecutive serialized histories share an initial segment, the reusable prefix can now extend through that segment. Appending a list element may replace a closing delimiter, so the reusable region is the actual common token prefix, not necessarily the complete previous field. This example requires no prediction of the next advice. It exploits the part of the next request that is already known.

Cross-call-site sharing introduces another boundary. Two sites may receive identical leading bindings but have different headers. A site-specific header before those bindings prevents sharing across sites at that point. Moving shared bindings ahead of that header can expose further reuse when the system prompts, serialization, and preceding tokens also match. Identical binding values alone are insufficient. These observations motivate learning a consistent layout that considers both sharing scope and evolution, while retaining labels and field contents.

3.3 Reuse opportunities need timely residency

A longer common prefix is useful only if its KV state is available. Recency alone cannot distinguish an intermediate result that another call will soon consume from a fresh diagnostic that the next attempt will replace. Nor does a recent access imply continued usefulness after its session terminates. Conversely, a prefix can remain useful throughout a tool execution despite receiving no accesses during that interval. Continuum addresses retention across such pauses through cache time-to-live, while CacheScout learns execution transitions to guide retention and prefetching [11, 16].

Binding heterogeneity supplies information beyond the identity of a future call. Predicting that implement will run again does not exploit its full potential. Its specification and plan can be reconstructed from the live session, the history contributes an existing prefix, and new advice remains unresolved. Re-layout can place these reconstructible inputs before the first unknown field, increasing the amount of KV that can be prepared before arrival. Call-transition and timing observations could determine which candidate prefixes are likely to be needed and when promotion should begin.

4 Design

To bridge the gap between the structured nature of programmatically generated prompts and the opaque treatment they receive at the serving layer, we propose IDONTKNOWITSNAME, an efficient serving engine for program-integrated LLM agents. IDONTKNOWITSNAME is a compiler-server co-design that identifies and exploits prompt heterogeneity for more efficient KV-cache management. On the compiler side, IDONTKNOWITSNAME performs static analysis to recover the

sharing scopes and lifetimes of prompt components, and proactively re-lays out prompt bindings to expose longer reusable prefixes. On the server side, IDONTKNOWITSNAME uses the recovered heterogeneity information to guide KV-cache scheduling, including cache prefetching, protection, and prediction-guided eviction. Together, these mechanisms enable the serving system to manage KV cache according to the expected reuse and lifetime of different prompt components rather than treating each request as an undifferentiated token sequence.

4.1 Adaptive Compiler Transformation

On compiler side, IDONTKNOWITSNAME uses static analysis and short runtime observation to decide the ordering of prompt components.

4.2 Predictive KV-Cache Scheduling

5 Related Work

LLM programming abstractions. MTP and its ByLLM implementation derive LLM operations from typed declarations and semantic information [2]. DSPy separates declarative signatures and modules from prompt construction and optimization [9]. NVIDIA Object-Oriented Agents embeds LLM behavior in Python objects [5], while SIGIL compiles skill documents into typed harnesses with explicit control flow [3]. These systems make program structure available during application construction. IDONTKNOWITSNAME studies how the resulting structure can inform serving: named bindings expose value evolution and sharing that token-prefix lookup alone does not identify. It consumes supplied templates or infers them from observed requests, and uses the recovered structure for cache management.

Memory-efficient inference and hierarchical caching. vLLM's PagedAttention improves KV allocation and sharing through paged memory management [10]. SGLang combines a structured programming interface with RadixAttention and cache-aware scheduling [18]. Sarathi-Serve uses chunked prefill to improve the throughput-latency tradeoff [1]. CachedAttention retains conversation KV in a hierarchy and overlaps data movement with computation [6]; Mooncake builds a distributed cache and scheduling architecture around KV reuse [13]. These mechanisms provide allocation, lookup, transfer, and execution support. IDONTKNOWITSNAME builds on SGLang and adds decisions about the serialized prefix and its future demand, using binding lifetimes to determine what can be reused and prepared.

Modular and non-prefix attention reuse. Prompt Cache explicitly defines reusable prompt modules through schemas and manages their positions for attention-state reuse [7]. CacheBlend targets inputs assembled from multiple text chunks, reusing cached states beyond the common prefix and selectively recomputing tokens to restore cross-chunk

interactions [14]. These works recognize that a prompt contains reusable components. IDONTKNOWITSNAME instead exposes common prefixes by reorganizing labelled bindings according to their observed evolution, then uses ordinary exact prefix caching on the resulting requests. It does not transplant KV tensors computed under a different preceding context. Exact cache reuse for a transformed request does not imply that the transformation preserves model outputs; the task-quality evaluation addresses that separate question.

Workflow-aware agent serving. KVFlow represents an agent schedule as an Agent Step Graph and uses distance to future execution for node-level eviction and overlapped prefetching [12]. PBKV predicts several future agent invocations in dynamic workflows using historical executions and current context, then conservatively guides eviction and prefetching [17]. CacheScout learns agent transitions online without a predefined workflow graph [16]. Continuum uses bounded retention through KV-cache time-to-live to preserve continuity across tool calls [11]. Pythia exposes workflow semantics through a serving interface to drive speculative cache management, lookahead scheduling, and autoscaling [15].

These systems establish the value of anticipating agent execution, including managing individual cache nodes and shared prefixes. IDONTKNOWITSNAME's distinction is the coupling of that anticipation with the structure inside generated prompts. It learns which bindings are constant, copied, extended, or volatile; uses this information to change the reusable prefix; and reconstructs the known portion of predicted requests to guide preparation and retention. Thus, workflow prediction supplies the likely destination and time of reuse, while binding analysis supplies the content and boundary of the reusable prefix.

6 Conclusion

The inherent structure and heterogeneity of programmatically generated prompts create opportunities for optimization. IDONTKNOWITSNAME identifies and exploits these opportunities rather than treating each prompt as a monolithic unit, thereby unlocking proactive optimizations that improve performance and efficiency. Our evaluation demonstrates that IDONTKNOWITSNAME's methodology successfully creates a more efficient LLM serving system.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 117–134.
- [2] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothynathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM*

- on *Programming Languages* 9, OOPSLA2 (2025), 1176–1204. doi:10.1145/3763092
- [3] Jayanaka L. Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. 2026. SIGIL: Compiling Agent Skills into Typed Harnesses. arXiv:2607.27309 [cs.SE] <https://arxiv.org/abs/2607.27309>
- [4] DSPy Contributors. 2026. ChatAdapter: DSPy API Reference. Online documentation. <https://dspace.ai/api/adapters/ChatAdapter/> Accessed September 14, 2026.
- [5] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schüller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. 2026. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. arXiv:2607.20709 [cs.AI] <https://arxiv.org/abs/2607.20709>
- [6] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [7] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. 325–338. https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html
- [8] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. 2020. HoVer: A Dataset for Many-Hop Fact Extraction And Claim Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3441–3460.
- [9] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [11] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS]
- [12] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. arXiv:2507.07400 [cs.DC] <https://arxiv.org/abs/2507.07400>
- [13] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [14] Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 16 pages. doi:10.1145/3689031.3696098
- [15] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. arXiv:2604.25899 [cs.MA] <https://arxiv.org/abs/2604.25899>
- [16] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhua Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving. arXiv:2608.14624 [cs.AI]
- [17] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. arXiv:2605.06472 [cs.LG] <https://arxiv.org/abs/2605.06472>
- [18] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.